==========

GETHIRED AI — PROJECT SUMMARY

Adaptive AI Interview Coach for Freshers

Built for the AMD Developer Hackathon (ACT II), "Unicorn" track

==========

1. PROBLEM STATEMENT

Most interview-prep platforms treat every candidate the same way: a fixed bank of generic questions, asked in a fixed order, regardless of who's answering. They don't read the candidate's resume, don't adjust based on how the candidate is actually performing, and don't explain why a question was asked or why a score was given. Freshers walk away with a number but no real understanding of what went wrong, which concepts they're weak in, or how a real interviewer would have read their performance.

The gap this project targets: the distance between "you scored 60%" and "here's specifically what to fix" is where most prep tools fail freshers, who don't yet have the pattern-recognition that comes from sitting through dozens of real interviews.

2. SOLUTION AND CORE THINKING

GetHired AI reads a candidate's resume, conducts a structured 8-question interview (HR -> Technical -> Hiring Manager), and continuously adapts its questioning strategy in real time based on performance — the way a real interviewer would. At the end it generates a recruiter-style report explaining not just the score, but *why* the interview unfolded the way it did.

The explicit design decision from the start: the differentiator is NOT "uses an LLM." It's that the system behaves like a real interviewer — it understands the resume, adapts questions to live performance, selects project questions by role-relevance ranking, tracks strengths/weaknesses across the whole interview, and explicitly explains why it changed strategy partway through (surfaced to the user as an "Interview Strategy Timeline"). This is what moves it from "AI chatbot" to "adaptive AI interviewing system."

Round order (HR -> Technical -> Hiring Manager) is fixed and never changes. What adapts is difficulty, follow-ups, which project gets discussed, and topic selection within rounds.

Scoring: 8 questions, 5 dimensions each (technical accuracy, communication, confidence, problem solving, relevance), 10 points/dimension, 50 points/question, 400 points total.

3. ARCHITECTURE

Four cooperating agents, orchestrated as a LangGraph state machine (chosen deliberately over a plain prompt chain, since the interview genuinely IS a state machine — the Strategy Agent's decision drives a conditional edge, not just the next prompt in a sequence):

Resume Analyzer -> Interview Agent <-> Strategy Agent -> Feedback Agent

- Resume Analyzer: parses the resume into a structured CandidateProfile (skills, projects ranked by role-relevance, education, certifications). Every other agent reads from this.

- Interview Agent: asks the next question in the current round's persona (HR / Technical / Hiring Manager), honoring whatever the Strategy Agent just directed (difficulty, follow-up, a specific project/topic to focus on). Never scores — that's the Feedback Agent's job.

- Strategy Agent: the USP. Never talks to the candidate. After every scored answer, decides how to adapt the NEXT question and logs an observation + decision, which powers the Strategy Timeline. Example behavior: if a candidate struggles on DSA but their resume shows a strong AI/RAG project, it redirects the next question to that project instead of continuing to drill the weak area.

- Feedback Agent: scores each answer on 5 dimensions, generates feedback

and a suggested improvement, detects weak/strong topic patterns across the interview, and builds the final report + hiring recommendation.

Shared state: one central `InterviewState` Pydantic object (backend/core/state.py) flows through every graph node. Agents never call each other directly — they only read/write this shared state, which is what keeps the LangGraph wiring clean.

Dual mode, selectable per-session and per-agent (via `agent_modes`):
- `mock` — fully offline, no API calls, instant. NOT scripted/canned — it evaluates whatever the candidate actually typed via deterministic heuristics (keyword/concept coverage + answer depth), so a blank or off-topic answer genuinely scores low.
- `llm` — real calls to Fireworks AI's OpenAI-compatible endpoint, model accounts/fireworks/models/gpt-oss-20b (open-weight, Apache 2.0).

Tech stack: Python / FastAPI / LangGraph / Pydantic on the backend, React (Vite) on the frontend, Fireworks AI for LLM inference.

4. DEVELOPMENT TIMELINE

The repo started as an empty scaffold: directory structure and file names existed (backend/agents/*.py, backend/core/*.py) but every file was a 0-byte stub, with only a standalone test_connection.py proving the Fireworks API key and endpoint worked.

Build order:

- Phase 0 — Dependencies (requirements.txt: openai, langgraph, fastapi, uvicorn, pypdf, pytest, httpx, etc.)
- Phase 1 — backend/core/state.py: the full InterviewState schema (Pydantic v2, with operator.add reducers on accumulating list channels so LangGraph nodes append rather than overwrite history/strategy_log)
- Phase 2 — backend/core/llm_client.py: single Fireworks entry point, complete_json() for structured JSON output with fence-stripping and one retry on parse failure
- Phase 3 — The four agents + backend/mocks/data.py (scripted candidate profile and per-question rubrics used by mock mode)
- Phase 4 — backend/core/graph.py: the LangGraph StateGraph, using interrupt()/Command(resume=...) to pause the graph at each question and resume with the candidate's typed answer
- Phase 5 — backend/main.py: FastAPI session lifecycle and turn-by-turn API
- Phase 6 — React (Vite) frontend: upload screen, interview loop, strategy timeline, final report

Each phase was verified in isolation (unit-level Python checks) before wiring into the next, and the full pipeline was run end-to-end through the compiled LangGraph (interrupt/resume across all 8 questions) before touching the API layer.

5. PROBLEMS FACED AND HOW THEY WERE SOLVED

This section is the honest retrospective — real bugs hit during development,
in the order they surfaced.

--- 5.1 Mock mode scored answers without reading them ----------

PROBLEM: Early mock mode returned a fixed, canned score/feedback per question index regardless of what the candidate actually typed — caught by the user testing the offline build and noticing scores looked "random" and suggested answers ignored their input.

ROOT CAUSE: `_mock_eval(index)` only used the question index as a lookup key into a static dict; the `answer` parameter was accepted but never inspected.

FIX: Rebuilt mock scoring as a real (if simple) heuristic: per-question rubrics of expected concepts/keywords, matched case-insensitively against the candidate's actual answer text, combined with an answer-depth/length signal. Blank or off-topic answers now genuinely score near zero; strong, on-topic answers score high. This also uncovered a second bug: the Strategy Agent's mock decision was still hardcoded to redirect at a fixed question index regardless of the real computed score — fixed by driving strategy decisions off the actual last-answer score, and gating the "redirect" decision on whether a redirect is actually possible at that point in the round layout (otherwise the Strategy Timeline could log an adaptation that couldn't happen, e.g. right as the interview moves into the Hiring Manager round).

--- 5.2 "Failed to fetch" in llm mode ----------

PROBLEM: Switching a session to real `llm` mode produced a generic "Failed to fetch" error in the browser.

INVESTIGATION: Reproduced directly against the live API rather than guessing. Raw Fireworks connectivity worked fine (test_connection.py succeeded). The actual failure was inside resume_analyzer._llm(): with no resume text supplied, the model sometimes returned a field with the wrong JSON type (e.g. `education` as a list instead of a string). The code constructed a Pydantic CandidateProfile directly from the model's raw JSON with no validation, so a single malformed field threw an unhandled ValidationError that propagated into a bare, unhelpful 500 response.

FIX: Added coercion helpers (safe_str, safe_str_list, safe_float, safe_int) at the LLM/external-API boundary in llm_client.py, and applied them across every agent that builds a Pydantic model from raw LLM JSON (resume analyzer, feedback agent, strategy agent, interview agent). Malformed fields now degrade to sane defaults instead of crashing the request.

--- 5.3 The whole server froze during llm-mode calls ----------

PROBLEM: While diagnosing 5.2, discovered a second, more serious issue: each LLM call is a synchronous, blocking HTTP request, called directly inside `async def` FastAPI route handlers. During a slow call (5-50 seconds observed, high variance), this blocked the single-threaded event loop entirely — no other request, including unrelated sessions' health checks, could be served until it returned.

FIX: Wrapped the two GRAPH.invoke() call sites in `asyncio.to_thread`, moving the blocking work off the event loop. Verified live: fired a /health request

while a 30-second llm-mode call was in flight — it now returns in ~20ms instead of hanging behind the slow call.

--- 5.4 Turns felt "broken" because they take 20-40 seconds ----------
PROBLEM: Every answer submission chains THREE sequential LLM calls before responding — score the answer, decide the next strategy, generate the next question — each depending on the previous one's output, so they can't run in parallel. Measured: ~8.6s + 15.1s + 5.5s per turn. With no progress indication, the UI just looked frozen for up to 40+ seconds.
DECISION POINT: The obvious latency fix (merge the Feedback + Strategy LLM calls into one prompt) was explicitly considered and rejected — it would have blurred the boundary between the two agents this project's whole pitch is built around (four *distinct*, cooperating agents). The user chose to keep the architecture separate and instead fix the perceived-broken feeling.
FIX: Rebuilt graph execution around GRAPH.stream(stream_mode="updates") instead of GRAPH.invoke(), which yields a signal each time a node completes. Added a GET /sessions/{id}/progress endpoint backed by an in-memory stage tracker, and a frontend polling loop (every 600ms) showing the live stage ("Scoring your answer...", "Deciding what to ask next...", "Preparing your next question..."). One bug caught during verification: the first version announced each stage AFTER its node finished, i.e. "Scoring your answer..." only appeared once scoring was already done — fixed by shifting the labels to announce each node's *successor* as it starts, with the very first stage announced eagerly by the caller.

--- 5.5 No evaluation of llm-mode output quality ------------------------------

PROBLEM: The entire automated test suite only exercised mock mode. Nothing verified that llm mode's actual model output was any good — whether scores were sane, whether the Strategy Agent's signature redirect behavior held up against a real model rather than a hardcoded mock heuristic.

FIX: Built a separate `eval/` harness (deliberately NOT part of the fast pytest suite, since real calls cost API credits and take minutes): fixtures.py (blank/weak/strong synthetic answers per question, reusing the mock rubric's reference answers as ground truth), state_builders.py (minimal InterviewState construction for direct agent calls, bypassing the graph), metrics.py (pure, LLM-free checks: score bounds, monotonicity, question validity, and the two headline behavioral checks — does the Strategy Agent redirect to the strongest project after a weak technical score, does it escalate difficulty after a strong one), and run_eval.py (a CLI that runs these against the real API and writes a JSON + Markdown report).

VERIFIED LIVE: a real run against Fireworks passed 9/9 checks, including both headline behavioral checks — confirming the model itself, unprompted by any hardcoded rule, makes the same adaptive call the mock heuristic hard-codes. This is the evidence that the project's core USP claim holds against the real model, not just the scripted demo.

--- 5.6 UI felt static / no sense of progress ---------------------------------

PROBLEM: The interview screen showed only "Question X / 8" as plain text; no visual sense of position within the interview, no loading feedback beyond a static label, generic Vite-template styling left over from scaffolding.

FIX: Added a visual progress stepper (3 round groups x question dots, filling

in as answered), a live word counter under the answer box, an animated loading-dots indicator, a circular SVG score ring and colored recommendation pill on the final report, and mobile-responsive breakpoints. Verified by actually driving the running app: no chromium-cli/Playwright was pre-installed in this environment, so Playwright was installed into a scratchpad (not the project's own dependencies) and used to drive a full headless-Chromium session through the entire interview, screenshotting each stage. One test-script bug caught in the process: a screenshot fired mid-way through a CSS fade-in animation, producing a washed-out image that looked like a rendering bug but was actually just a race in the verification script itself — fixed by waiting for the animation to settle before capturing.

--- 5.7 Leftover scaffold clutter ----------

PROBLEM: An empty backend/models/ directory from the original scaffold (never used — the real models live in backend/core/state.py), orphaned Vite template assets (react.svg, vite.svg, hero.png, an unused icons.svg) with zero references anywhere in the rewritten frontend, a generic "React + Vite" boilerplate README with no project-specific content, and an un-gitignored .DS_Store.

FIX: Verified each file was genuinely unreferenced (grep across the whole source tree) before deleting anything, removed them, added a macOS section to .gitignore, and re-ran the full test suite + frontend build + lint afterward to confirm nothing broke.

--- 5.8 Real API key committed to a file meant to be public ----------

PROBLEM (the most serious issue found): while verifying setup instructions for documentation, discovered .env.example — the template file meant to be committed to git, as opposed to the gitignored .env — contained what looked like a real Fireworks API key rather than a placeholder.

INVESTIGATION: Confirmed the key exactly matched the real, working key in the actual (properly gitignored) .env file. Confirmed .env.example was NOT gitignored. Checked git history: the key was committed in the very first commit ("Initial commit: GetHired AI"). Checked the configured remote (github.com/Rk8661/gethired-ai) and confirmed local HEAD matched origin/main's commit hash exactly — meaning the commit containing the real key had already been pushed.

RESPONSE: Stopped other work immediately to flag this. Fixed the local file to use a placeholder. Explicitly did NOT attempt to rewrite git history or force-push without being asked, since that rewrites shared history other clones would need to be redone from. Recommended immediate key rotation at Fireworks as the only action that actually neutralizes a key already pushed to a remote (editing the local file alone does not undo the exposure). The user rotated the key directly on the Fireworks dashboard.

SECONDARY FIX: while touching this file, also found and corrected a second, smaller bug — .env.example documented a `VITE_API_URL` variable, but the frontend code actually reads `import.meta.env.VITE_API_BASE`. Setting the documented variable name would have silently done nothing.

--- 5.9 Dependency version drift broke the graph at runtime ----------

PROBLEM: A routine question ("which langgraph version does the code expect") led to discovering the code no longer ran. backend/core/graph.py constructed `JsonPlusSerializer(allowed_msgpack_modules=[...])` — verified working earlier in development against langgraph==1.2.7. Running the test suite against whatever was now installed threw `TypeError: __init__() got an unexpected keyword argument 'allowed_msgpack_modules'`.

INVESTIGATION: `pip show langgraph` now reported version 0.6.11, not 1.2.7, and the Python interpreter inside the project's own venv had changed from 3.13.5 to 3.9.6 at some point — strong evidence the venv had been recreated outside the development conversation. Ruled out a stale-pip-index theory by upgrading pip from 21.2.4 to 26.0.1 and re-checking: langgraph's real, current latest version on PyPI is genuinely 0.6.11; 1.2.7 was not resolvable at all. Inspected the installed JsonPlusSerializer's actual constructor signature directly and confirmed the parameter simply does not exist in the langgraph-checkpoint version (2.1.2) that ships with 0.6.11.

FIX: Simplified graph.py to use the default MemorySaver() with no custom serializer configuration (the gating mechanism the removed parameter controlled doesn't exist in this checkpoint version in the first place), and re-pinned requirements.txt to langgraph==0.6.11 — the real, currently resolvable version — rather than continuing to reference an unobtainable one.

VERIFIED: full pytest suite (28/28), a live backend HTTP run through all 8 questions, a frontend build, and a full Playwright browser-driven run through the entire interview UI with zero console errors, end to end.

--- 5.10 Docker setup had two bugs that would only surface at actual runtime --

CONTEXT: Docker support (backend/Dockerfile, frontend/Dockerfile, docker-compose.yml) was added directly to the repo. No Docker daemon is available in the development sandbox this project was built in, so a true `docker-compose up` could not be executed — the following were found and fixed by careful static inspection of exact COPY-order and networking semantics, not by trial-and-error against a running build.

PROBLEM 1 (secrets baked into an image layer): neither Dockerfile had a matching .dockerignore. The backend Dockerfile's build context is the repo root and does `COPY . .` — with no ignore file, that would copy the real .env (containing actual credentials) directly into the image. Pushing such an image to any registry would re-expose a secret in almost the same way as the earlier .env.example incident (5.8), just baked into image layers instead of git history.

PROBLEM 2 (wrong platform binaries): the frontend Dockerfile runs `RUN npm install` and only afterward does `COPY . .`. Without an ignore file, that second COPY would overwrite the container's freshly-installed (Linux) node_modules with the host machine's local node_modules — including host-platform native binaries (oxlint, lightningcss, rolldown bindings) that don't run inside a Linux container.

PROBLEM 3 (browser can't reach the API): docker-compose.yml set the frontend's VITE_API_BASE to `http://backend:8000` — a hostname Docker only resolves container-to-container. Since this is a client-side React SPA, API calls are made by the browser running on the host machine, which cannot resolve that hostname at all. Every API call would have failed once the stack was actually brought up.

FIX: added a root .dockerignore (excludes .env, venv/, frontend/node_modules, .git, caches) and a frontend/.dockerignore (excludes node_modules/, dist/), and corrected VITE_API_BASE to `http://localhost:8000` (the host-published port). YAML syntax of docker-compose.yml was validated programmatically. CAVEAT: none of this was confirmed against an actual `docker-compose up` — that still needs to be run by whoever has Docker available, as the one piece of verification that couldn't be done in this environment.

6. PERFORMANCE

Automated tests (mock mode + pure logic, zero network calls):

28 tests, full suite runs in well under 1 second.

Mock mode (offline heuristic scoring):

Effectively instant — a full 8-question interview completes in milliseconds end to end.

LLM mode (real Fireworks calls, model: gpt-oss-20b):

Individual call latency: 3-52 seconds observed, high variance even for identical prompts.

Per-turn latency (score + strategize + next question, 3 sequential calls): roughly 20-40 seconds typical, driven by real measurement during development (8.6s + 15.1s + 5.5s in one measured run).

Session creation (resume analysis + first question, 2 calls): ~7-15s typical, though a single outlier run took 52.6s, illustrating the variance.

reasoning_effort="low" is used for all structured-output calls (score/ strategy/question generation are extraction-style tasks, not deep reasoning) — measurably faster than the default effort level with no observed loss in output validity.

A 90-second client-side timeout bounds worst-case wait so a stuck call fails with a clear error rather than hanging indefinitely.

LLM eval harness cost/time (real API calls, for reference):

Small eval (3 questions x 3 fixture kinds, 1 repeat): ~14 calls, roughly 3.5-7 minutes wall-clock, well under $0.05 in API credits.

Full eval (all 8 questions x 3 fixture kinds, 1 repeat): ~34 calls, roughly 8-17 minutes, still under $0.20.

A live full run scored 9/9 checks passed, including both headline behavioral checks (strategy redirect on weak score, difficulty escalation on strong score).

7. CURRENT STATE / WHAT WORKS

- Full 8-question interview flow works end-to-end in both mock and llm mode, verified live via direct HTTP calls and via a real browser driven through the actual React UI.
- LangGraph state machine with interrupt/resume correctly pauses at each question and resumes with the candidate's typed answer.
- Strategy Agent's signature adaptive redirect (weak technical answer -> pivot to the candidate's strongest, most relevant resume project) verified

working against both the mock heuristic and the real LLM.

- FastAPI backend does not block under concurrent load during slow llm-mode calls.
- Frontend shows live progress during slow turns instead of a static spinner, plus a visual question-progress stepper, score ring, and recommendation pill on the final report.
- Fast pytest suite (28 tests) covers mock-mode pipeline logic and the eval harness's own pure scoring functions; a separate, opt-in eval/ CLI verifies llm-mode output quality against the real model without slowing down or costing money on every normal test run.
- No known exposed secrets remain in the working tree (the leaked key has been rotated by the project owner).
- Docker support exists (backend/Dockerfile, frontend/Dockerfile, docker-compose.yml) and passed static review (correct YAML, no secrets or wrong-platform binaries baked into images, correct browser-reachable API URL) — see 5.10. NOT yet confirmed against an actual `docker-compose up`, since no Docker daemon was available in this development environment; that is the one remaining verification step for whoever runs it next.

==========